

Entregable Final - Programación IV

I. INDICACIÓN

Campo	Descripción
Asignatura	Programación IV
Unidad	Bloque 1 - Frameworks / Bloque 3 - Patrones de Diseño / Bloque 4 - Calidad de Sistemas
Tema	Desarrollo de un sistema de información genérico con CRUD e integración de IA local mediante Ollama, asistido por OpenCode
Duración estimada	8 horas (3 sesiones de laboratorio)
Modalidad	Práctica individual o en parejas
Puntaje total	100 puntos
Fecha de entrega	29 de Septiembre
Herramientas	Python 3.11+, Django 5.x, Ollama, SQLite, Git, VSCodium, OpenCode, terminal Linux (Debian 12), navegador web

Entregables:

- **Carpeta del proyecto Django** completa y funcional (código fuente, `requirements.txt`, archivo `.env.example` si se usan variables de entorno).
- **Documento en formato Markdown** (`informe.md`) que documente todo el proceso: diseño, implementación asistida por OpenCode, integración con Ollama, pruebas realizadas, resultados obtenidos y evidencias de las consultas realizadas con OpenCode.
- **Documento PDF** (`informe.pdf`) generado a partir del informe Markdown (puede usarse Pandoc, VS Code Markdown PDF, o similar).
- Los entregables deben comprimirse en un solo archivo `.zip` con la siguiente estructura:

```
Apellido_Nombre_EntregableFinal_ProgramacionIV.zip
├── proyecto/
│   ├── (código del proyecto Django)
│   ├── requirements.txt
│   └── .env.example
├── informe.md
└── informe.pdf
```

II. MOMENTO

Contexto de la Actividad

Imagine que forma parte del equipo de desarrollo de una empresa que necesita digitalizar la gestión de su información. Se le ha asignado crear un **sistema de información genérico** que permita administrar registros de cualquier entidad (por ejemplo, productos, clientes, libros, empleados, etc.) mediante operaciones CRUD. Además, el sistema debe integrar un **modelo de inteligencia artificial local** ejecutado con **Ollama** para que los usuarios puedan realizar consultas en lenguaje natural sobre los datos registrados, obteniendo respuestas generadas a partir de la información real almacenada en la base de datos.

Para acelerar el desarrollo y mejorar la calidad del código, se utilizará **OpenCode**, un agente de codificación con IA de código abierto que funciona en la terminal y que permitirá asistir en la generación de código, depuración, documentación y refactorización. El informe final deberá documentar tanto el proceso de desarrollo como las interacciones realizadas con OpenCode.

El objetivo es demostrar que es posible construir una aplicación web robusta con Django, siguiendo buenas prácticas de desarrollo, aplicando patrones de diseño y garantizando la calidad del software, sin depender de servicios de IA en la nube.

Escenario

Rol	Responsabilidad	Entregable	Requisitos
Desarrollador	Implementar el sistema CRUD e integrar Ollama con asistencia de OpenCode	Proyecto Django funcional + informe	CRUD completo, reportes, chat con IA local, documentación, uso de OpenCode
Entidad	Objeto de gestión (genérico)	Modelo de datos	Campos definidos por el estudiante (mínimo 6 campos)
IA local	Responder consultas sobre los datos	Integración con Ollama	Modelo local (ej. qwen2.5:1.5b o similar)
OpenCode	Asistente de codificación	Interacciones documentadas	Sesiones, prompts y respuestas documentadas

III. DESARROLLO

Consigna General

El estudiante deberá **desarrollar una aplicación web con Django** que implemente un **CRUD completo para una entidad genérica** de su elección (por ejemplo: productos, clientes, libros, empleados, etc.), e **integrar un modelo de IA local mediante Ollama** para permitir consultas en lenguaje natural sobre los datos registrados. Todo el desarrollo deberá ser **asistido por OpenCode**, documentando las interacciones realizadas con esta herramienta.

El sistema debe cumplir con los siguientes requisitos:

- CRUD completo:** crear, leer, actualizar y eliminar registros de la entidad elegida.
- Validaciones:** campos obligatorios, unicidad de código/identificador, valores no negativos donde corresponda.
- Reportes predefinidos:** al menos 5 reportes que resuman la información (ej. total de registros, el más caro/barato, registros con valores bajos, etc.).
- Chat con IA local:** formulario donde el usuario escribe preguntas en lenguaje natural; Django envía la pregunta y los datos relevantes a Ollama; Ollama genera una respuesta basada exclusivamente en los datos reales.

5. **Restricción de respuestas:** la IA no debe inventar información; si no hay datos suficientes, debe indicarlo.
6. **Manejo de errores:** si Ollama no está disponible, el sistema debe informar adecuadamente.
7. **Buenas prácticas:** organización del código en apps, uso de patrones de diseño (Factory, Strategy, Observer, etc.), validaciones, mensajes de éxito/error.
8. **Documentación:** el proyecto debe incluir un `README.md` con instrucciones de instalación y ejecución, y un archivo `DOCUMENTACION.md` con las decisiones técnicas y la arquitectura.
9. **Uso de OpenCode:** documentar las sesiones de trabajo con OpenCode, los prompts utilizados y las respuestas obtenidas, así como su impacto en el desarrollo.

No se solicitarán capturas de pantalla. El estudiante deberá documentar los **procesos realizados** y los **resultados obtenidos** mediante descripciones textuales, fragmentos de código, salidas de comandos, y explicaciones de los resultados de las pruebas.

Punto 1: Diseño e implementación del CRUD (30 pts)

Subpunto	Descripción
1.1	Definir la entidad a gestionar y sus campos (mínimo 6 campos, incluyendo un identificador único). Documentar el modelo de datos en el informe.
1.2	Implementar el modelo en Django, las migraciones correspondientes y el panel de administración personalizado. Utilizar OpenCode para asistir en la generación del código.
1.3	Desarrollar las vistas y plantillas para el CRUD completo (lista, detalle, creación, edición y eliminación). Aplicar validaciones y mensajes de éxito/error.
1.4	Implementar al menos 5 reportes predefinidos (pueden ser vistas o comandos) que resuman la información de la entidad.

Producto esperado:

- **En el informe Markdown:** descripción del modelo de datos, fragmentos de código relevantes (`models.py` , `views.py` , `forms.py`), comandos ejecutados (`makemigrations` , `migrate` , `runserver`), explicación de los resultados de cada reporte, y **evidencia de las interacciones con OpenCode** (prompts y fragmentos de código generados).

Punto 2: Integración con IA local mediante Ollama (40 pts)

Subpunto	Descripción
2.1	Instalar y configurar Ollama en el entorno local. Descargar un modelo (ej. <code>qwen2.5:1.5b</code>). Documentar los comandos utilizados.
2.2	Implementar la comunicación entre Django y Ollama. Crear una función/servicio que reciba la pregunta del usuario, obtenga los datos de la base de datos, construya un contexto y envíe la consulta a la API local de Ollama (<code>http://localhost:11434</code>). Utilizar OpenCode para asistir en la implementación.
2.3	Desarrollar un formulario de chat en una vista de Django que permita al usuario escribir preguntas y recibir respuestas generadas por Ollama.
2.4	Implementar la restricción de respuestas: la IA solo debe usar los datos proporcionados. Si no hay información suficiente, debe responder con un mensaje predefinido.
2.5	Manejar el caso en que Ollama no esté disponible, mostrando un mensaje adecuado al usuario.

Producto esperado:

- **En el informe Markdown:** comandos de instalación de Ollama y descarga del modelo, fragmentos de código del servicio de integración (`services/ollama_service.py`), del formulario y la vista del chat, ejemplos de preguntas y respuestas obtenidas (texto plano, sin capturas), y **evidencia de las interacciones con OpenCode** para esta integración.

Punto 3: Calidad, patrones y documentación (30 pts)

Subpunto	Descripción
3.1	Aplicar al menos dos patrones de diseño en el proyecto (por ejemplo: Factory para servicios, Strategy para reportes, Observer con señales de Django). Documentar su implementación.
3.2	Escribir pruebas unitarias para al menos dos funcionalidades críticas (ej. validación de código único, cálculo de reportes). Ejecutar las pruebas y documentar los resultados.
3.3	Documentar el proyecto: <code>README.md</code> con instrucciones de instalación y ejecución, <code>DOCUMENTACION.md</code> con las decisiones técnicas y arquitectura.
3.4	Generar el archivo <code>requirements.txt</code> con <code>pip freeze > requirements.txt</code> y un archivo <code>.env.example</code> si se usan variables de entorno.
3.5	Documentar el uso de OpenCode: sesiones realizadas, prompts utilizados, respuestas obtenidas y su impacto en el desarrollo.

Producto esperado:

- **En el informe Markdown:** descripción de los patrones aplicados (con fragmentos de código), resultados de las pruebas unitarias (salida de `python manage.py test`), enlaces a los archivos de documentación del proyecto, y **evidencia de las interacciones con OpenCode** para la refactorización, pruebas y documentación.

IV. RÚBRICA DE EVALUACIÓN

Criterio	Puntaje	RECEPTIVO (0%)	RESOLUTIVO (50%)	AUTÓNOMO (75%)	ESTRATÉGICO (100%)
1. Diseño e implementación del CRUD	30 pts	No implementa el CRUD o la entidad no tiene campos suficientes.	Implementa el CRUD pero con errores o sin validaciones.	Implementa el CRUD completo con validaciones y reportes básicos.	Implementa el CRUD con validaciones robustas, bien diseñado, documentación clara, proceso de desarrollo y el uso de OpenCode.
2. Integración con IA local (Ollama)	40 pts	No integra Ollama o no logra obtener respuestas.	Integra Ollama pero las respuestas no se basan en los datos o hay errores frecuentes.	Integra Ollama correctamente, las respuestas usan los datos y maneja errores básicos.	Integra Ollama correctamente, restringe respuestas a los datos, maneja errores, documenta el proceso y la calidad de las respuestas, evidencia en OpenCode.
3. Calidad, patrones y documentación	30 pts	No aplica patrones, no escribe pruebas ni documenta.	Aplica un patrón o escribe pruebas mínimas, documentación escasa.	Aplica dos patrones, escribe pruebas funcionales y documenta el proyecto.	Aplica dos patrones, pruebas exhaustivas, documentación clara, decisiones técnicas, <code>requirements.txt</code> y <code>.env</code> , evidencia en OpenCode estas tareas.

V. MATERIAL DE APOYO

Nombre de la carpeta: Material_EntregableFinal_ProgramacionIV

Contenido sugerido:

- Guía de instalación de Ollama en Linux.
- Documentación oficial de Django (modelos, vistas, formularios, admin).
- Tutorial de integración de Django con APIs externas (requests).
- Guía de patrones de diseño en Python.
- Ejemplo de `.env.example` y uso de variables de entorno con `python-dotenv`.
- Herramientas para generar PDF desde Markdown (Pandoc, VS Code Markdown PDF, etc.).
- Guía de instalación y uso de OpenCode (CLI y TUI).
- Ejemplos de prompts para OpenCode en tareas de desarrollo Django.

VI. EJE TRANSVERSAL

Tecnologías emergentes y adaptabilidad digital – El estudiante demuestra capacidad para adaptarse a nuevas herramientas de IA local (Ollama, OpenCode) y resolver problemas prácticos en un entorno de desarrollo moderno.

Investigación y pensamiento crítico – El estudiante investiga, selecciona y evalúa modelos de IA, analiza su estructura, identifica puntos de mejora y propone soluciones creativas para integrarlos en una aplicación Django, utilizando OpenCode como asistente.

VII. ODS

ODS	Descripción	Vinculación con la actividad
ODS 4: Educación de calidad	Garantizar una educación inclusiva, equitativa y de calidad.	La actividad fomenta el aprendizaje práctico y autónomo en tecnologías de IA local y desarrollo web, preparando al estudiante para el mundo laboral.
ODS 8: Trabajo decente y crecimiento económico	Promover el crecimiento económico sostenido, inclusivo y sostenible.	El desarrollo de habilidades en Django, IA local y OpenCode contribuye a la empleabilidad y al emprendimiento tecnológico.
ODS 9: Industria, innovación e infraestructura	Construir infraestructuras resilientes, promover la industrialización inclusiva y sostenible.	El uso de herramientas open source como Django, Ollama y OpenCode impulsa la innovación y el desarrollo de soluciones de IA accesibles y soberanas.

VIII. FORMATO DE ENTREGA

Estructura del entregable

El entregable final es un archivo `.zip` con la siguiente estructura:

```
Apellido_Nombre_EntregableFinal_ProgramacionIV.zip
├── proyecto/
│   ├── manage.py
│   ├── (apps del proyecto)
│   ├── requirements.txt
│   └── .env.example
├── informe.md
└── informe.pdf
```

Requisitos del documento Markdown (`informe.md`)

- **Portada:** Título, nombre del estudiante, asignatura, docente, fecha.
- **Estructura clara:** Utilizar encabezados (`#` , `##` , `###`) para organizar las secciones (Punto 1, Punto 2, Punto 3, Uso de OpenCode).

- **Código y comandos:** Utilizar bloques de código (`bash` , `python` , `json`) para mostrar todos los comandos ejecutados y su salida. **No se requieren capturas de pantalla**; en su lugar, describir textualmente los resultados obtenidos.
- **Evidencia de OpenCode:** Incluir una sección específica que documente las sesiones de trabajo con OpenCode: prompts utilizados, respuestas obtenidas, fragmentos de código generados y su impacto en el desarrollo.
- **Reflexión técnica:** Incluir una sección final donde el estudiante explique las dificultades encontradas, cómo las resolvió y qué aprendió.
- **Citas y referencias:** Incluir enlaces a la documentación oficial de Django, Ollama y OpenCode.

Requisitos del documento PDF (`informe.pdf`)

- Generado a partir del informe Markdown utilizando una herramienta como Pandoc, VS Code Markdown PDF, o similar.
- Debe incluir todo el contenido del informe Markdown, con un formato legible.

Requisitos de la carpeta del proyecto (`proyecto/`)

- El proyecto debe **ejecutarse correctamente** con las instrucciones documentadas en el `README.md` .
- El archivo `requirements.txt` debe generarse con `pip freeze > requirements.txt` dentro del entorno virtual activado.
- El archivo `.env.example` debe listar las variables de entorno necesarias (ej. `OLLAMA_URL` , `OLLAMA_MODEL` , `SECRET_KEY` , `DEBUG`).
- El proyecto debe incluir al menos un archivo de pruebas (`tests.py` o carpeta `tests/`) con las pruebas unitarias implementadas.
- El proyecto debe incluir un archivo `DOCUMENTACION.md` con las decisiones técnicas y la arquitectura.
- El proyecto debe incluir un directorio `opencode/` o similar con las transcripciones de las sesiones de OpenCode (si es posible exportarlas) o, en su defecto, un archivo `OPENCODE.md` con las interacciones documentadas.